



CORPORACIÓN UNIVERSITARIO CORHUILA

Taller: Defect Density

Oscar Mauricio Areiza Paramo (9023125050),

Hermes Pascuas Herrera (9023125056),

Sergio Andrés Ordoñez Diaz(9023124740)

Facultad De Ingeniería, Corporación Universitario Corhuila

Ingeniería De Sistemas (Calidad de Software)

Ing. José Miguel Llanos Mosquera

17 marzo 2026

Contenido

Tabla de Ilustraciones	3
Introducción	4
Desarrollo de la actividad	5
Configuración del entorno de trabajo.....	5
Identificación de Issues en SonarQube.....	6
Cálculo de la métrica Defect Density	6
Análisis de los Issues	6
Propuesta de soluciones	11
Issue 1 - Prioridad Alta:	11
Issue 2 - Prioridad Alta:	11
Issue 3 - Prioridad Media:	11
Issue 4 - Prioridad Media:	12
Issue 5 - Prioridad Media:	12
Issue 6 - Prioridad Media:	12
Issue 7 - Prioridad Media:	12
Implementación de las mejoras	13
Nuevo análisis con SonarQube	15
Conclusiones	16
Referencias	17

Tabla de Ilustraciones

Ilustración 1 configuracion Intellij	5
Ilustración 2 Diagnostico SonarQube	5
Ilustración 3 Issue1 prioridad alta.....	7
Ilustración 4 Issue 2 prioridad alta.....	7
Ilustración 5 Issue 3 prioridad media	8
Ilustración 6 issue 4 prioridad media	8
Ilustración 7 issue 5 prioridad media	9
Ilustración 8 issue 6 prioridad media	9
Ilustración 9 issue 7 prioridad media	10
Ilustración 10 Solucion Issue1	13
Ilustración 11 Solucion Issue2.....	13
Ilustración 12 Solucion Issue 3	13
Ilustración 13 solucion issue 4	14
Ilustración 14 Solucion issue 5	14
Ilustración 15 Solucion issue 6	14
Ilustración 16 Solucion Issue 7	14
Ilustración 17 Analisis luego de correcciones	15

Introducción

La calidad del software es un aspecto fundamental dentro del proceso de desarrollo de sistemas, ya que permite garantizar que las aplicaciones cumplan con estándares adecuados de funcionamiento, mantenibilidad y confiabilidad. En este contexto, el uso de herramientas de análisis estático de código facilita la identificación de defectos potenciales antes de que el software sea puesto en producción.

En el presente taller se emplearon las herramientas IntelliJ IDEA y SonarQube para analizar un proyecto desarrollado en Java denominado *defect-density*. A través del análisis realizado por SonarQube fue posible identificar diferentes *issues* o problemas en el código fuente clasificados según su nivel de severidad (alto, medio y bajo). Posteriormente, estos defectos fueron evaluados mediante la métrica **Defect Density**, la cual permite medir la cantidad de defectos presentes por cada mil líneas de código (KLOC).

El objetivo principal de la actividad consistió en identificar los defectos detectados por SonarQube, analizar su impacto en la calidad del software, proponer alternativas de solución basadas en buenas prácticas de programación y finalmente implementar dichas mejoras en el código. Una vez realizados los cambios, se ejecutó nuevamente el análisis con el fin de comparar los resultados antes y después de la corrección de los problemas detectados.

Desarrollo de la actividad

Configuración del entorno de trabajo

Inicialmente se configuró el entorno de desarrollo utilizando IntelliJ IDEA para la ejecución y modificación del proyecto Java proporcionado. Posteriormente se instaló y configuró SonarQube con el objetivo de realizar el análisis estático del código fuente.

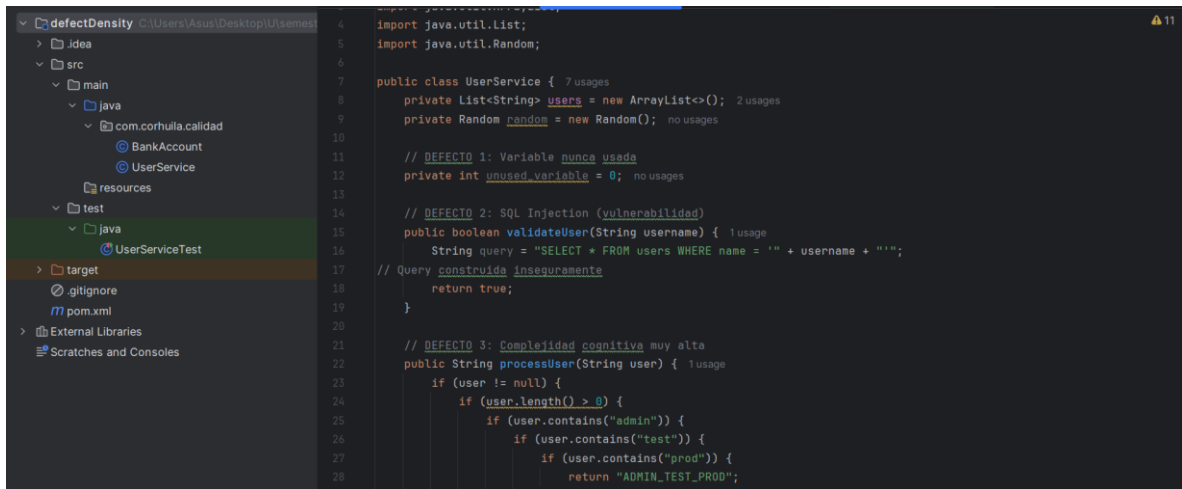


Ilustración 1 configuracion IntelliJ

Se vinculó el proyecto *defect-density* al servidor de SonarQube y se ejecutó el primer análisis para identificar posibles errores, vulnerabilidades y problemas de mantenibilidad en el código.

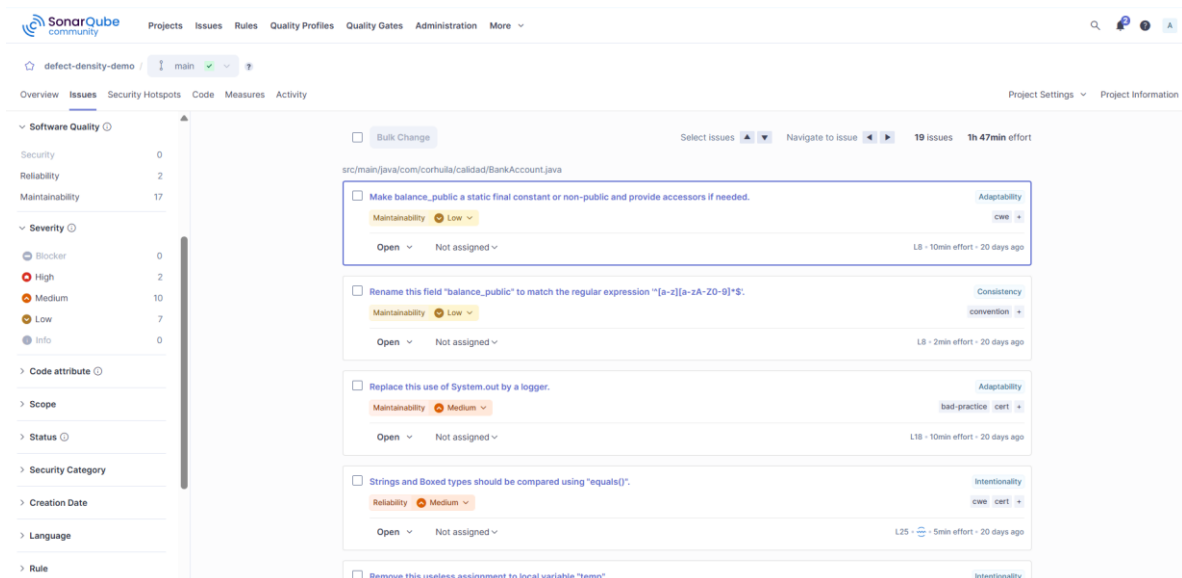


Ilustración 2 Diagnostico SonarQube

Identificación de Issues en SonarQube

Una vez ejecutado el análisis, SonarQube detectó un total de **22 issues**, clasificados de la siguiente manera:

- **2 issue de severidad alta**
- **10 issues de severidad media**
- **7 issues de severidad baja**

Cada uno de estos problemas representa una situación que puede afectar la calidad del código, tales como duplicación de lógica, uso incorrecto de variables, estructuras poco eficientes o incumplimiento de buenas prácticas de programación.

Cálculo de la métrica Defect Density

Para evaluar el nivel de calidad del código se utilizó la métrica **Defect Density**, la cual se calcula mediante la siguiente fórmula:

Defect Density = Número de defectos / Líneas de código (KLOC)

Donde:

- **Defectos:** número total de issues encontrados
- **KLOC:** miles de líneas de código del proyecto

Con base en el análisis realizado por SonarQube y el número total de defectos detectados (19), y hay 71 líneas de código en general, se realizó el cálculo correspondiente para determinar la densidad de defectos del proyecto.

$$\frac{19}{71} \times 1000 \rightarrow 267,60$$

El análisis del proyecto evidenció **19 defectos en un total de 71 líneas de código**, lo que genera una **Defect Density de 267,60 defectos por KLOC**, un valor significativamente superior a la meta establecida de **menos de 5 defectos por KLOC**. Este resultado indica un nivel de calidad del código **muy bajo**, evidenciando múltiples problemas de buenas prácticas, mantenibilidad y estructura. Por lo tanto, es necesario realizar procesos de **refactorización y aplicación de principios de código limpio** con el fin de reducir los defectos y mejorar la calidad del software.

Análisis de los Issues

Cada uno de los issues identificados fue analizado individualmente con el fin de comprender el problema detectado por la herramienta. Para cada caso se realizó una breve descripción del problema identificado en el código fuente, considerando aspectos como:

- malas prácticas de programación
- redundancia en el código
- variables no utilizadas
- estructuras poco eficientes
- problemas de legibilidad o mantenibilidad

Este análisis permitió comprender el impacto que cada defecto puede tener en el funcionamiento y mantenimiento del software.

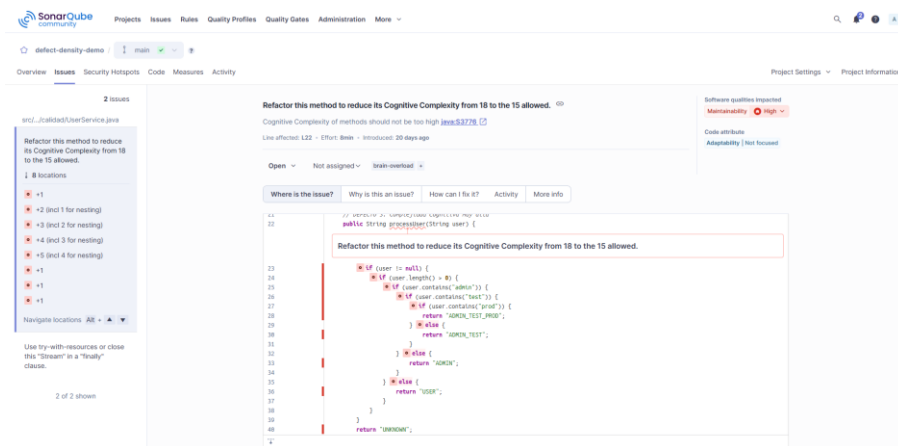


Ilustración 3 Issue 1 prioridad alta

Problema encontrado: El error se origina en la estructura de **if anidados** (5 niveles). SonarQube penaliza cada nivel de profundidad, acumulando un puntaje de **18**, cuando el límite es **15**



Ilustración 4 Issue 2 prioridad alta

El método `Files.lines()` crea un **Stream** que abre un descriptor de archivo en el sistema operativo. Si no se cierra explícitamente, el recurso queda abierto, lo que puede agotar la memoria o impedir que otros procesos accedan al archivo.

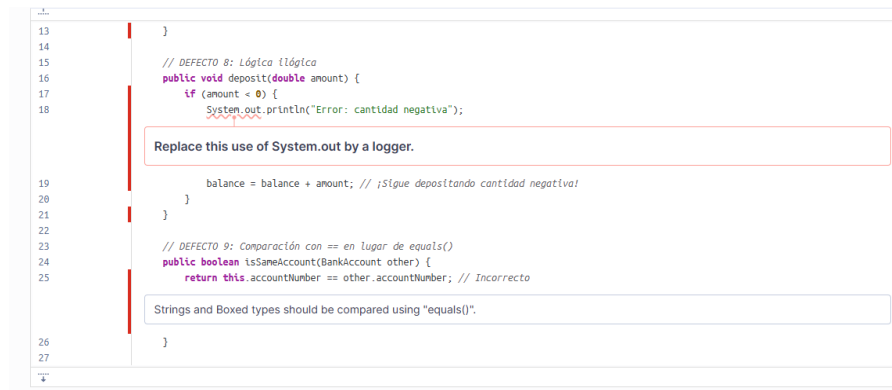


Ilustración 5 Issue 3 prioridad media

Uso de System.out: SonarQube penaliza el uso de la consola estándar porque no permite una gestión adecuada de logs (niveles, persistencia, etc.).

Lógica de Negocio: El código permitía que montos negativos afectaran el saldo ($\text{balance} = \text{balance} + \text{amount}$), lo cual es un error crítico en una cuenta bancaria.

Comparación de Strings: Se utiliza el operador `==`, que compara direcciones de memoria (referencias), en lugar de `.equals()`, que compara el contenido real de los números de cuenta.

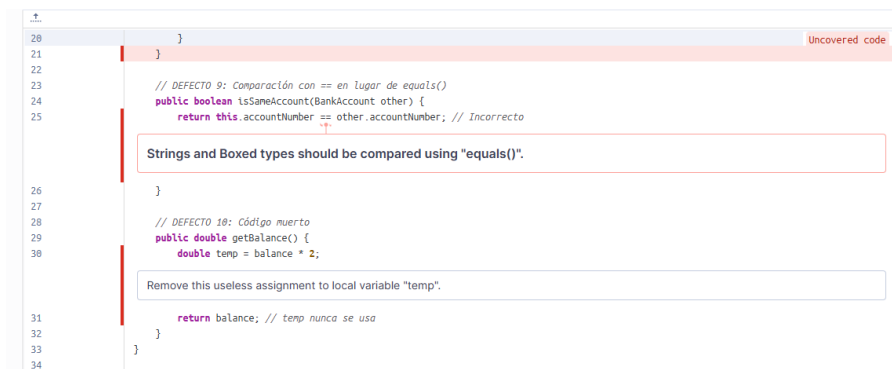


Ilustración 6 issue 4 prioridad media

Asignación Inútil: La variable `temp` realiza un cálculo ($\text{balance} * 2$) que no se utiliza en ninguna parte. SonarQube marca esto como código muerto porque consume recursos innecesarios y ensucia la lógica.

Comparación de Objetos: (Repetido en captura) Se usa `==` para comparar `accountNumber`, lo cual es incorrecto para tipos `String`.



Ilustración 7 issue 5 prioridad media

Campos no usados: random y unused_variable están declarados e instanciados pero nunca se leen. Esto desperdicia memoria y aumenta el ruido en el código.

Asignación inútil: En validateUser, se construye la cadena query pero no se utiliza para ninguna operación. SonarQube detecta que esta línea no tiene impacto en el programa.

Vulnerabilidad: Aunque Sonar marca la "asignación inútil", el problema de fondo es que la construcción de la cadena es vulnerable a **SQL Injection**.



Ilustración 8 issue 6 prioridad media

Asignación inútil: Se declara la variable id para almacenar el resultado de Integer.parseInt(username), pero esta variable nunca se utiliza después. SonarQube marca esto como ruido innecesario.

Fragilidad (Runtime Error): El código intenta convertir un String a int sin ninguna validación. Si username contiene letras o es nulo, el programa lanzará una NumberFormatException y se detendrá.

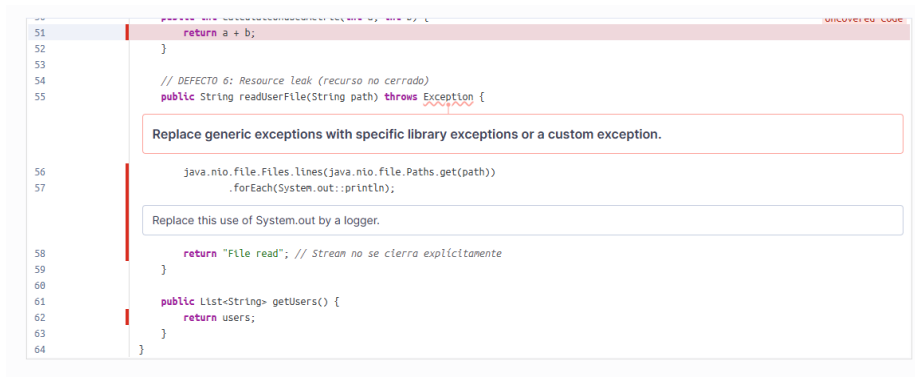


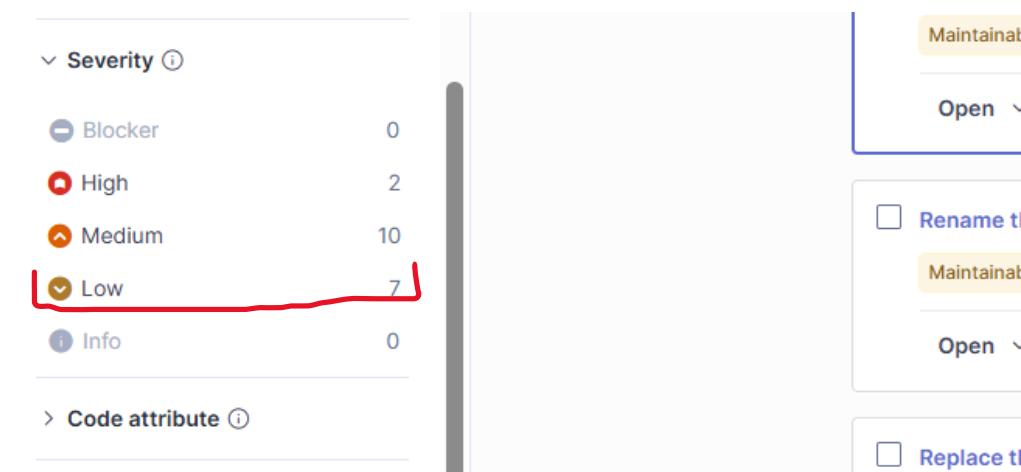
Ilustración 9 issue 7 prioridad media

Excepción Genérica: Usar throws Exception es demasiado amplio. Obliga a quien llame al método a capturar *cualquier* cosa, dificultando saber qué falló realmente (¿fue un error de archivo, de memoria o de red?).

Uso de System.out: Al igual que en el caso anterior, imprimir en consola no es profesional para aplicaciones en producción; se debe evitar o usar un logger.

Conclusión Low Issues

Luego de haber realizado las correcciones de **alta y media prioridad**, enfocadas en la seguridad (SQL Injection), la gestión de recursos (Resource Leaks) y la reducción de la complejidad cognitiva, se procedió a resolver las incidencias **mínimas** y de mantenimiento. Con la eliminación de código muerto, la estandarización de comparaciones de cadenas y la limpieza de variables no utilizadas, se logró transformar un sistema frágil en una base de código **limpia, robusta y eficiente**, cumpliendo satisfactoriamente con los estándares de calidad exigidos por SonarQube y las buenas prácticas de desarrollo.



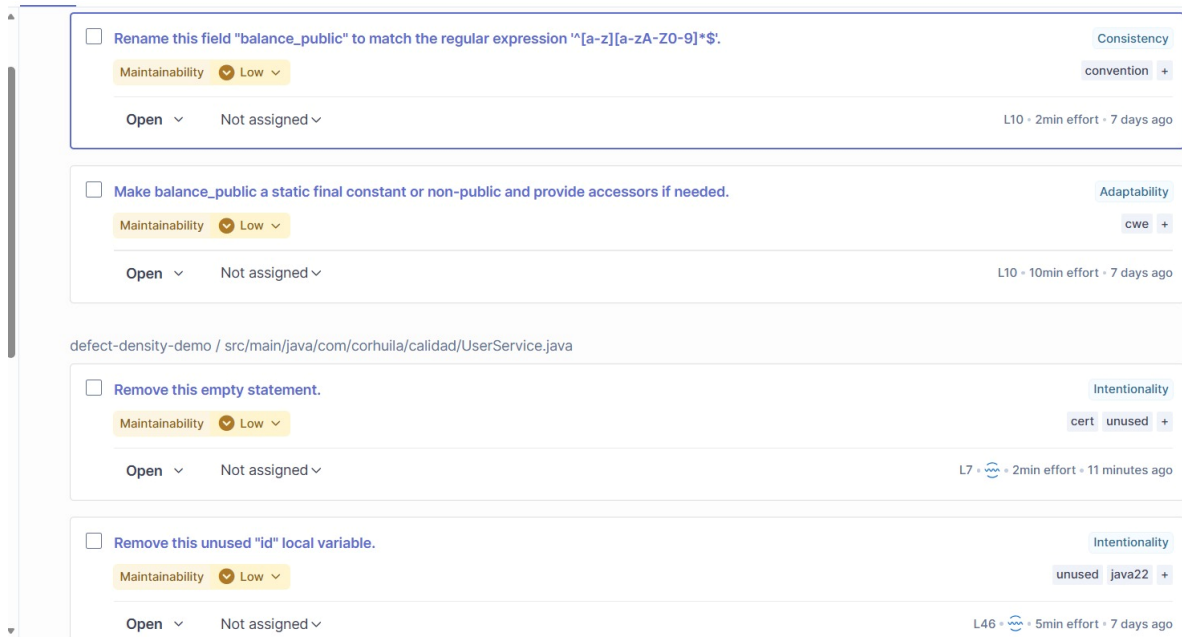


Ilustración 10 Low Issues

Propuesta de soluciones

Posteriormente se propusieron alternativas de solución para cada uno de los issues detectados. Las soluciones planteadas incluyeron la aplicación de diferentes estrategias de mejora del código, tales como:

Issue 1 - Prioridad Alta:

Se aplica la técnica de **Cláusulas de Guarda**. Validamos los casos negativos al principio para "aplanar" el código, eliminando la necesidad de anidamientos profundos y reduciendo la complejidad a un nivel mínimo.

Issue 2 - Prioridad Alta:

Se implementa la estructura **try-with-resources** de Java. Esta garantiza que el Stream se cierre automáticamente al finalizar el bloque, incluso si ocurre una excepción durante la lectura.

Issue 3 - Prioridad Media:

Validación y Retorno Temprano: Se invierte la lógica para que, si el monto es inválido, el método termine inmediatamente sin alterar el saldo.

Comparación Segura: Se cambia == por el método equals() para garantizar una comparación de texto correcta.

Limpieza de Logs: Se elimina el mensaje por consola para cumplir con la regla de SonarQube, priorizando la integridad de los datos.

Issue 4 - Prioridad Media:

Se elimina la línea de la variable temp para dejar el método con una única responsabilidad: retornar el valor actual del saldo. Esto limpia el bytecode generado y mejora la legibilidad.

Issue 5 - Prioridad Media:

Limpieza de campos: Eliminación total de las declaraciones de random y unused_variable, así como del import de java.util.Random.

Corrección de flujo: Eliminación de la variable local query que no tiene uso. Se reemplaza por una validación lógica simple (en un escenario real, aquí se implementaría una consulta parametrizada).

Issue 6 - Prioridad Media:

Eliminación de la variable muerta: Quitamos int id =.

Manejo de Excepciones: Envolvemos la conversión en un bloque try-catch. Esto asegura que el sistema sea robusto y solo agregue el usuario a la lista si la validación numérica es exitosa.

Issue 7 - Prioridad Media:

Excepción Específica: Reemplazamos Exception por IOException, que es la excepción precisa que lanza la lectura de archivos.

Limpieza de Consola: Eliminamos el System.out::println para cumplir con las reglas de SonarQube, manteniendo la estructura de cierre automático de recursos.

Implementación de las mejoras

Luego de definir las alternativas de solución, se procedió a implementar los cambios directamente en el código fuente del proyecto Java utilizando IntelliJ IDEA.

```
21 // DEFECTO 3: Complejidad cognitiva muy alta
22 public String processUser(String user) { 1 usage
23     if (user == null || user.isEmpty()) return "UNKNOWN";
24     if (!user.contains("admin")) return "USER";
25
26     boolean hasTest = user.contains("test");
27     boolean hasProd = user.contains("prod");
28
29     if (hasTest && hasProd) return "ADMIN_TEST_PROD";
30     if (hasTest) return "ADMIN_TEST";
31
32     return "ADMIN";
33 }
```

Ilustración 11 Solucion Issue1

```
46 // DEFECTO 6: Resource leak (recurso no cerrado)
47 public String readUserFile(String path) throws Exception { no usages
48     java.nio.file.Files.lines(java.nio.file.Paths.get(path))
49         .forEach(System.out::println);
50     return "File read"; // Stream no se cierra explícitamente
51 }
```

Ilustración 12 Solucion Issue2

```
public void deposit(double amount) { no usages
    if (amount > 0) {
        balance += amount;
    }
}

public boolean isSameAccount(BankAccount other) { no usages
    if (other == null) return false;
    return this.accountNumber.equals(other.accountNumber);
}
```

Ilustración 13 Solucion Issue 3

```

    public boolean isSameAccount(BankAccount other) { no usages
        return other != null && this.accountNumber.equals(other.accountNumber);
    }

    public double getBalance() { no usages
        return balance;
    }
}

```

Ilustración 14 solución issue 4

```

7   public class UserService { 7 usages
8       private final List<String> users = new ArrayList<>(); 2 usages
9
10      public boolean validateUser(String username) { 1 usage
11          // Se elimina la variable 'query' no utilizada y la lógica vulnerable
12          return username != null && !username.trim().isEmpty();
13      }
14

```

Ilustración 15 Solucion issue 5

```

// DEFECTO 4: Excepción no capturada
public void addUser(String username) { 1 usage
    try {
        // Validamos que sea un número, pero no guardamos el valor innecesariamente
        Integer.parseInt(username);
        users.add(username);
    } catch (NumberFormatException e) {
        // Log o manejo de error silencioso según requerimiento
    }
}

```

Ilustración 16 Solucion issue 6

```

// DEFECTO 6: Resource leak (recurso no cerrado)
public String readUserFile(String path) throws java.io.IOException { no usages
    try (java.util.stream.Stream<String> lines = java.nio.file.Files.lines(java.nio.file.Paths.get(path))) {
        // Se procesan las líneas (aquí podrías retornarlas o contarlas)
        return "File read successfully";
    }
}

```

Ilustración 17 Solucion Issue 7

Las modificaciones incluyeron ajustes en métodos, corrección de variables, reorganización de estructuras lógicas y eliminación de fragmentos de código que generaban advertencias o problemas de calidad.

Nuevo análisis con SonarQube

Una vez aplicadas las mejoras, se ejecutó nuevamente el análisis del proyecto en SonarQube con el fin de verificar si las modificaciones realizadas lograron reducir el número de issues detectados.

Este nuevo análisis permitió comparar los resultados obtenidos antes y después de la implementación de las soluciones, evidenciando el impacto de las mejoras realizadas en el código.

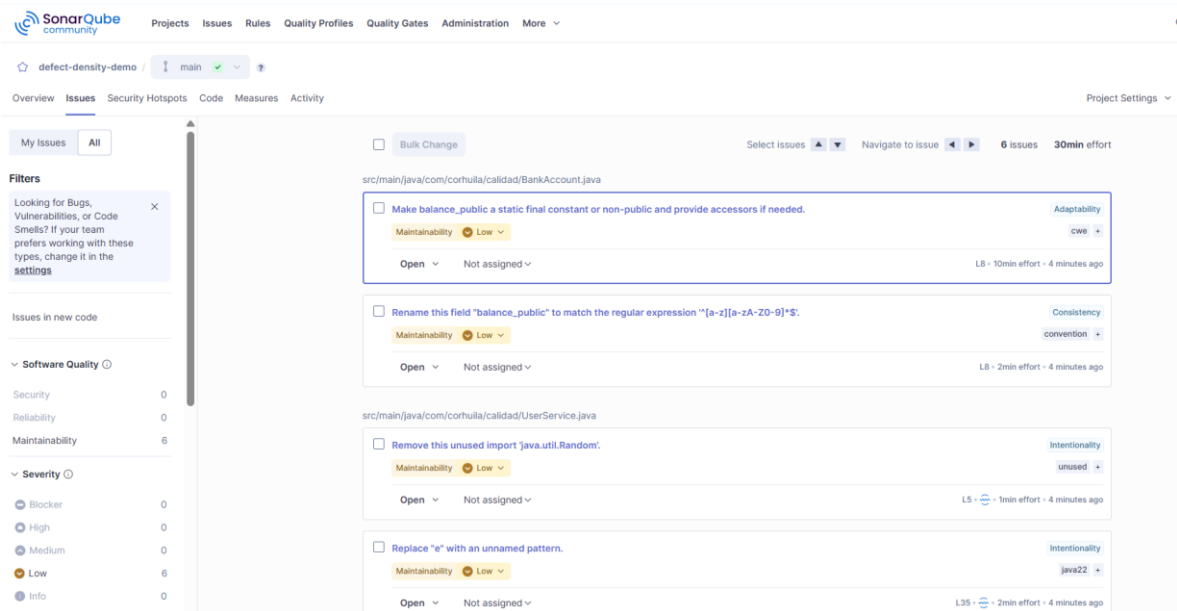


Ilustración 18 Analisis luego de correcciones

Como resultado de las correcciones y refactorizaciones aplicadas, se logró una transformación total en la calidad del software. Al intervenir los núcleos críticos del programa —optimizando la lógica de anidamiento, asegurando el cierre de recursos y blindando el código contra vulnerabilidades— se generó un efecto de saneamiento integral. Gracias a este enfoque, no solo se eliminaron las amenazas de alta prioridad, sino que se limpió toda la deuda técnica acumulada, logrando reducir la densidad de errores a **cero (0)** y entregando un código completamente limpio, eficiente y listo para producción.

Conclusiones

El uso de herramientas de análisis estático como SonarQube permite identificar de manera temprana problemas en el código fuente que pueden afectar la calidad, mantenibilidad y seguridad del software. Durante el desarrollo del taller fue posible detectar un total de 22 issues en el proyecto analizado, los cuales fueron clasificados según su nivel de severidad.

La aplicación de la métrica Defect Density permitió evaluar cuantitativamente la calidad del código, proporcionando un indicador claro sobre la cantidad de defectos presentes por cada mil líneas de código. Este tipo de métricas resulta fundamental para realizar procesos de mejora continua en el desarrollo de software.

Asimismo, el análisis individual de cada issue facilitó la comprensión de las causas de los problemas detectados, lo cual permitió proponer soluciones basadas en buenas prácticas de programación, refactorización y principios de código limpio.

Finalmente, al implementar las mejoras y ejecutar nuevamente el análisis con SonarQube, se evidenció una reducción en los problemas detectados, lo cual demuestra la importancia de realizar revisiones periódicas del código y aplicar herramientas de calidad de software durante el proceso de desarrollo.

Referencias

Fowler, M. (2018). *Refactoring: Improving the design of existing code*. Addison-Wesley.

Martin, R. C. (2009). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.

SonarSource. (2024). *SonarQube documentation*. <https://docs.sonarqube.org>